toolchain → Compilation Processess

source files → [toolchain] → executable file
- windows → .exe
- Minix (linux | Mac) → .out
- Embedded target → .hex / .elf

* toolchain → ① PreProcessor
  ② Compiler
  ③ Assembler → (Symbole table)
  ④ linker → collect all file in one file

* by toolchain You Can generate at specific Phase

↳ ① PreProcessor

file.c → [ ] → file.i

[gcc -E file.c -o file.i]

flag → PreProcessor only.

↳ ② Compiler

file.i → [Compiler] → file.asm / file.s / file.ss

gcc -S file.c -o file.asm

↳ ③ Assembler

file.asm → [Assembler] → file.o

gcc -c file.c -o file.o

↳ ④ linker

files.o, files.a, startupCode.o → [linker] → file.exe / .out / .elf / .hex

---

*gcc flags

↳ -Wall ↳ -Wextra ↳ -Werror

→ in cmd → gcc --help=warning

* optimize → control on level of optimize.
  ↳ -O <No> → [0] → [3] level
  ↳ -O fast ↳ -Os

* -std= → C_Code version → C89, C90, C99

---

* PreProcessor directives.

① any line have # Not take size from Memory

② the Replacement will be effect on Memory (will take size from Memory) Code Memory (ROM)

③ any directives (#) will be Handle by PreProcessor → expection → (#Pragma) ↳ compiler.

---

[1] include

* → #include will be include Header file only

* → #include → text Replacement for Header file content in the same line

Replacement #include Headefile → content

* → #include have two type

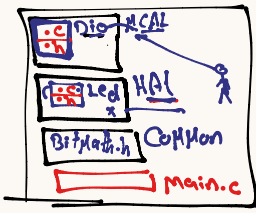
include header file From system lib [toolchain Path]
↳ use < >
will be search on this header file in tool chain Path

include the user define header file.
Use the ① " "
userdefine header file → " Path to file "

Asboulte Path
* will write the full Path
* Not used with team work

Relative Path
* write Path according to the file You Include

* Relative Path include



project
① main.c
#include "MCAL/Dio/Dio.h"
#include "HAL/Led/led.h"
#include "Common/BitMath.h"

② Led.c (../ → Back one step)
↳ BitMath.h → #include "../../Common/BitMath.h"
↳ Dio.h → #include "../../MCAL/Dio/Dio.h"

[2] define

object like Macro — Function like Macro

* create name for constant value or expression → to make your code Readability.

→ **object like Macro**

#define Name Value → const 'A'

* Your name of Macro must be unique.
* #define effect in the following line so we write it in header file or in file.c but After #include
* You can't write variable or function have name of Macro

*ex

```
#include <stdio.h>
#define size 5 →
int main()
{ int arr[size];
  for( i=0; i<size; i++);
  {
  =
  →int size =20; → compiler int 5=20;
  5 size =40 ; → 5=40;
```

#define Max size
print (Max);

→ **Function like Macro**

#define FunctionName (Input)   expression

→ Reg |= (1<<Bit)
→ x |= (1<<y);
No |= (1<< Index);

ex: → create Header file → BitMath.h

```
# define setBit (Reg, BitNo)  Reg |=(1<<BitNo)
```

→ main.c

```
#include "BitMath.h"
int main()         x |=(1<<5)
{
  setBit(x,5);
  SetBit(No,y);    No|=(1<<y);
  SetBit(z,3);
```

Compare between Normal function / Macro function

* Normal function → slow execution time.

```
int setBit(int Reg, int No)      #define setBit(Reg,No)
{ Reg |= (1<<No);  8 byte             Reg |=(1<<No)
  return Reg;                         No.
}
int main()                        int main()
{                                 {
  x=SetBit(x,5);   8 byte           setBit(x,5);
  y=SetBit(y,3);                    x|=(1<<5); → A
  z=SetBit(z,8);                    setBit(y,3);
                                    y|= (1<<3) → 4
                                    setBit(z,8);
                                    z|=(1<<8);
```

→ ① Slow execution time
→ ② Save Memory in Rom & RAM

→ Fast execution
↳ take memory. → Rom, RAM

---

*

[3] Condition Compilation

#if   #elif   #else   #endif   #ifndef   #ifdef

* if you used #if or #ifndef or #ifdef Must write #endif

* #endif → PreProcessor → if is finished

→ fire detection system
↳ any building

school — Max temp.
Company — Max temp
Factory — Max temp

waterPump

* Pre Configuration

config.h

```
#define building_type  1
```

→ if the #if true compiler

//school only

Program.c (Main.c)

```
#include "config.h"
int main()
{
#if building_type==1
  [ //school
    code for school ]
#elif building_type==2
  [ //Company
#elif building_type==3
  [ //factory.
#endif
```

#if #elif #endif

Config.h

```
#define type school
```

Private.h

```
#define school 1
#define company 2
#define factory 3

#if type == school
#elif type == company
#elif type == fac
#else
  #warning "Mess"
#endif default code
```

Program.c

```
#if type == school
  void set_maxtemp( ) ;
  void set_waterpumps ;
#endif

#if type == company

#endif
```

```
#else
  #error " message"
```

Difference between error / warning

- → warning → see wrong but it solve it
  - → default code
  - → generate executable file
- → error → see wrong line but can't solve it
  - → Not generate anything

PreProcessor